MAWE — final presentation script

Marco Perozo · the finished project · Phase 4 complete

Target: **8 minutes** of talking with the demo running inside it, then questions. Cut sections 4 and 6 if you only have five.

Have open before you start 1. A terminal in the project folder, large font, nothing else on screen. 2. `docs/reports/FINAL-REPORT.pdf`. 3. GitHub, on the Actions tab, so the green ticks are one click away.

Run the demo with

```
powershell -ExecutionPolicy Bypass -File scripts\demo.ps1
```

or double-click `demo.bat`. It pauses before every step and prints the line to say, so you are never typing while someone watches.

> Everything in > below is meant to be said out loud. Everything in *italics* is a stage direction. Do not read the italics.

---

0 · Opening — 20 seconds

> "Our project is the Multi-Agent AI Workflow Engine — MAWE. The one-line version: you give it a plain-English idea, and a team of AI agents plans it, builds it, reviews it, and fixes it until it works.
>
> Before I show you anything, one thing up front. Everything is built and tested, and two of our five hypotheses are answered by measurement. The other three compare AI models against each other, and **we have not run that comparison** — it needs a paid API key. The machinery for it is finished and validated. I'll come back to that at the end."

Say the limitation here, in the first twenty seconds. If you raise it, it reads as rigour. If he finds it, it reads as an oversight. This single paragraph is the most important thing in the script.

---

1 · The problem — 45 seconds

> "A normal AI coding assistant is one model answering one prompt. In a single pass, that model has to understand what you meant, design a solution, write the code, and check its own work. It usually produces something plausible — and it has nobody to catch what it missed.
>
> We split that into three specialists and a loop, the way a real team works. A **Product Manager** turns a vague request into a precise specification. A **Developer** writes the code against it. A **Code Reviewer** checks it, and if it fails, it goes back to the Developer with the

specific problems.

The research question is not 'can agents write code' — they can. It's whether splitting the work up produces **more correct** software than one model doing everything, and what that costs."

---

2 · The one design decision that matters — 45 seconds

"If the Reviewer is just another model reading code and giving an opinion, you've built an expensive way to get a second guess.

So the Product Manager doesn't only write acceptance criteria in English. It writes **runnable checks** — 'run this program with the argument one hundred C, and standard output must be exactly two-one-two point zero'. The Reviewer executes those in a sandbox before it forms a verdict.

Where the model's opinion and the program's actual behaviour disagree, **the measurement wins**. A model cannot talk its way past a failing program. That is the difference between this and asking the AI twice."

---

3 · Show it working — 2 minutes

Run step 2 of the demo. While the output scrolls:

"Watch the Product Manager write five acceptance criteria. The Developer implements it. Then the Reviewer runs it — and rejects it, for a specific reason: self-collision never ends the game.

Second iteration: the Developer fixes that exact thing. The second review finds only a naming nit — severity 'minor', which doesn't block — so the run is approved.

Two iterations, and the thing that ended the loop was a measurement, not an opinion."

---

4 · The hard part — 1 minute 15 *(the strongest CS content)*

"Wiring three prompts together is easy. Two things make it hard, and they are where the actual engineering is.

First: the loop might never stop. Two agents disagreeing is an infinite loop, and every cycle costs money — an unattended run could burn the whole API budget overnight. So there are five ways out. The subtle one is oscillation: if you only check 'is this the same as last time', code that alternates between two versions passes that check forever. We detect that separately."

Run step 3 — the stress test. It takes about two seconds.

"A hundred thousand randomised adversarial review sequences. None failed to terminate.

None exceeded their iteration cap.

Second: the conversation grows forever. By iteration twenty you're re-sending the entire history every time and paying for every token. So older iterations collapse into a structured log. Deliberately **not** by asking a model to summarise — a summariser that hallucinates would corrupt the Developer's only memory of what it already tried. At eighty iterations the prompt drops from fifty-two thousand tokens to three thousand, a ninety-four percent reduction, with no change in outcome."

---

5 · How we tested it with no API key — 1 minute 30

This question is coming whether or not you invite it. Invite it.

"You might reasonably ask how a project that has never called a real AI model can claim to be tested.

The answer is a distinction. **Testing the machinery around a model and testing the quality of a model are two different jobs.** The first needs no API key. The second does, and we haven't done it.

Every agent talks to one interface with a single method. A real provider implements it by calling Anthropic; our mock implements it by replaying a scripted answer. There is no test mode anywhere — the engine never imports the mock and cannot tell the difference. Swapping to a real model is one environment variable.

And the generated code is real. It's written to disk and executed in a subprocess, and we compare the actual bytes of output. No API involved in that at all.

Some things can **only** be tested this way. To prove the no-progress exit fires, something has to return byte-identical code twice. No real model will do that reliably — they're non-deterministic by design."

If he looks sceptical, run step 4. It demonstrates all of that in twenty seconds, and ends by stating what none of it proves.

---

6 · The benchmark and the comparison — 1 minute 15

"For the comparison we built a benchmark: twenty tasks in Python and JavaScript, with a hundred and eighty-one hidden tests the agents never see. That matters — it means the score doesn't depend on our own reviewing model. A run our Reviewer approved can still score zero.

And we measured the benchmark itself rather than assuming it was any good. Every task has a reference solution that passes all its hidden tests. A program that starts and does nothing solves zero of twenty. And when we inject deliberate single-line bugs into the reference solutions, the hidden tests catch ninety-one percent of them.

That last check earned its place — it found two real bugs in our own test suite. Every error-

path test was asserting 'exit status one', which an uncaught exception also produces, so a file of prose was scoring three out of seven on FizzBuzz."

Only run step 6 if you have time. If you do, say the warning sentence before the numbers appear:

> "These numbers come from a fixture with known answers, not a real model. They show the experiment pipeline works. They say nothing about any model's ability."

---

7 · Close — 40 seconds

> "So, where it stands. Four milestones, all met. The engine works, the benchmark is validated, the comparison apparatus is built and proven to detect a difference.
>
> Two hypotheses answered by measurement: the loop always terminates, and the memory compression works without changing outcomes.
>
> Three not answered: whether role separation actually produces more correct software, whether it covers requirements better, and whether it's worth the cost. Those need one sweep against a real model — two hundred and forty runs, about an hour of compute.
>
> What's missing is an API key, not code."

Then stop talking.

The three questions most likely to come back

"Why didn't you just get an API key?"

> "Budget — the proposal allowed fifty to a hundred and fifty dollars for the semester and we prioritised building something worth spending it on. The sweep is one command and the cost estimate prints before it runs."

"Isn't the mock just avoiding the hard part?"

> "It's the opposite — some guarantees can only be tested that way. No real model will return byte-identical code on demand, or refuse to approve anything fifty times in a row. Those adversarial cases are exactly what a safety guarantee depends on."

"What's the weakest part of this?"

> "Two things. The live model run, which I mentioned. And our comparison between the full engine and the self-refining baseline changes two variables at once — role separation and the sandbox-grounded review — so a difference couldn't be attributed to one of them. We report that rather than hide it. Separating them would need a fifth condition."

Answering that last one well is worth more than any part of the demo. It shows you understand what your own experiment can and cannot support, which is the thing being assessed.

If something breaks live

Don't debug in front of him. Say:

> "That's not behaving — let me show you the recorded result instead."

Then open `docs/reports/FINAL-REPORT.pdf` and the Actions tab. Every number in that report was generated from a measurement, and the green ticks prove the suite passes on three Python versions on machines that aren't this one.